# 4. HAFTA: Proje Yönetimi ve Planlanması

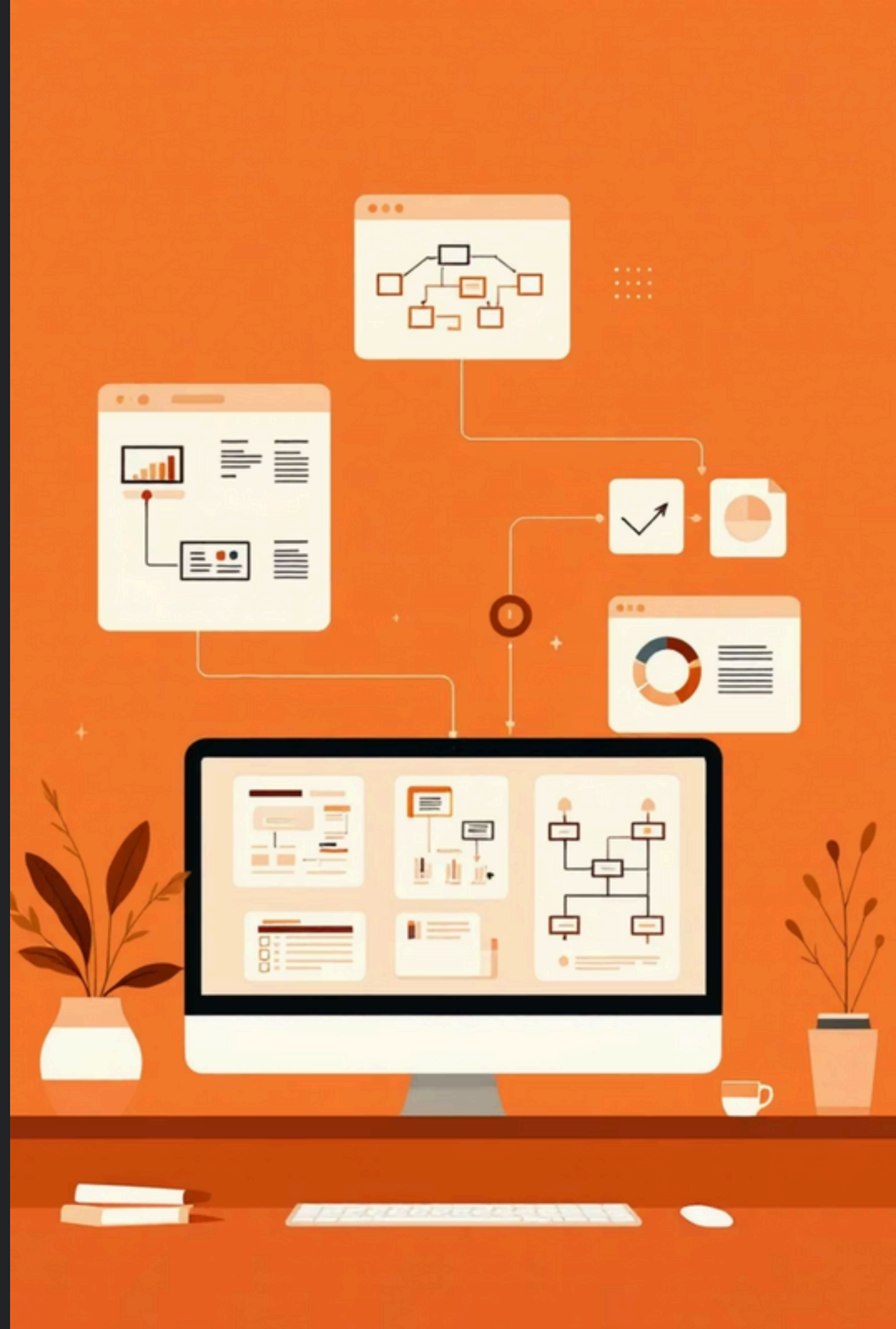
❖ Planlama Teknikleri ve Araçları (WBS, Gantt chart ve kaynak tahsisi)

❖ Takım Yönetimi ve İletişim Stratejileri (Roller, RACI matrisi ve motivasyon)



# Planlama Teknikleri ve Araçları

Proje yönetimi ve planlaması, yazılım projelerinin başarısını belirleyen temel unsurlardır. Bu süreç, projenin kapsamını, zamanını, maliyetini ve kaynaklarını organize eder; belirsizlikleri azaltır ve riskleri yönetir. Özellikle yazılım projelerinde, değişken gereksinimler ve ekip dinamikleri nedeniyle planlama kritik öneme sahiptir.



# l. WBS (Work Breakdown Structure - İş Kırılım Yapısı)

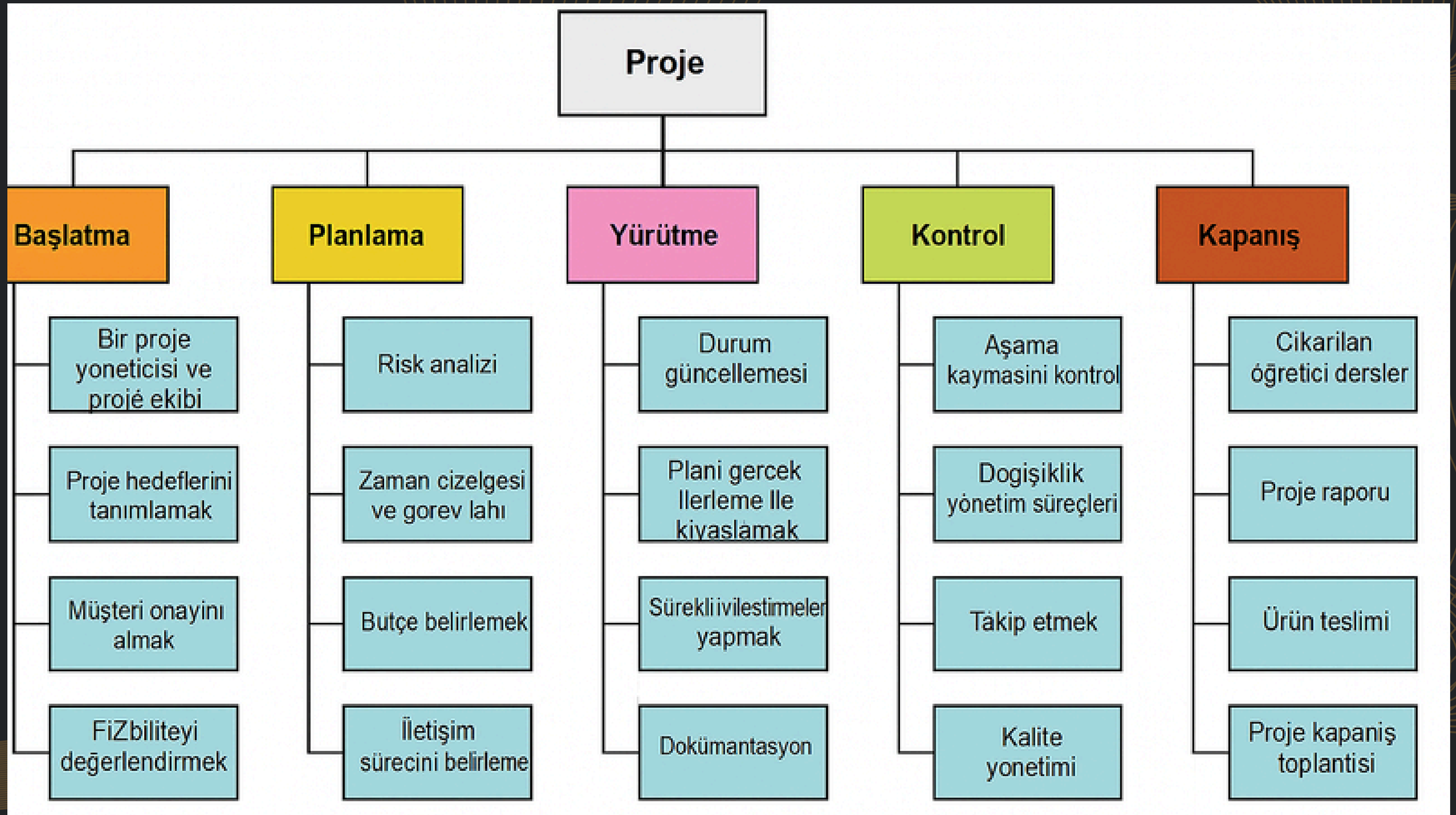
**WBS, projenin genel hedefine ulaşmak için iş kapsamını hiyerarşik bir yapıya bölen bir tekniktir. Proje, yönetilebilir alt görevlere (work packages) ayrılır; bu, kapsamı netleştirir, çakışmaları önler ve ilerlemeyi ölçmeyi kolaylaştırır. Yazılım mühendisliğinde, karmaşık kodlama ve entegrasyon süreçlerini parçalara ayırmak için idealdir. WBS, bir ağaç diyagramı veya liste şeklinde oluşturulur; en üst seviye proje hedefi, alt seviyeler detaylı görevlerdir. Faydası: Tahmin hatalarını %20-30 azaltır, ekip sorumluluklarını dağıtır.**

Ekip üyelerine net görevler verir

Bütçe ve zaman tahmini için temel sağlar

İlerlemeyi takip etmeyi kolaylaştırır

# l. WBS (Work Breakdown Structure - İş Kırılım Yapısı)



# Örnek: Bir web sitesi geliştirme projesi için WBS

## 1.0 Başlatma

(Proje Tanımı, Paydaş Analizi)

## 2.0 Tasarım

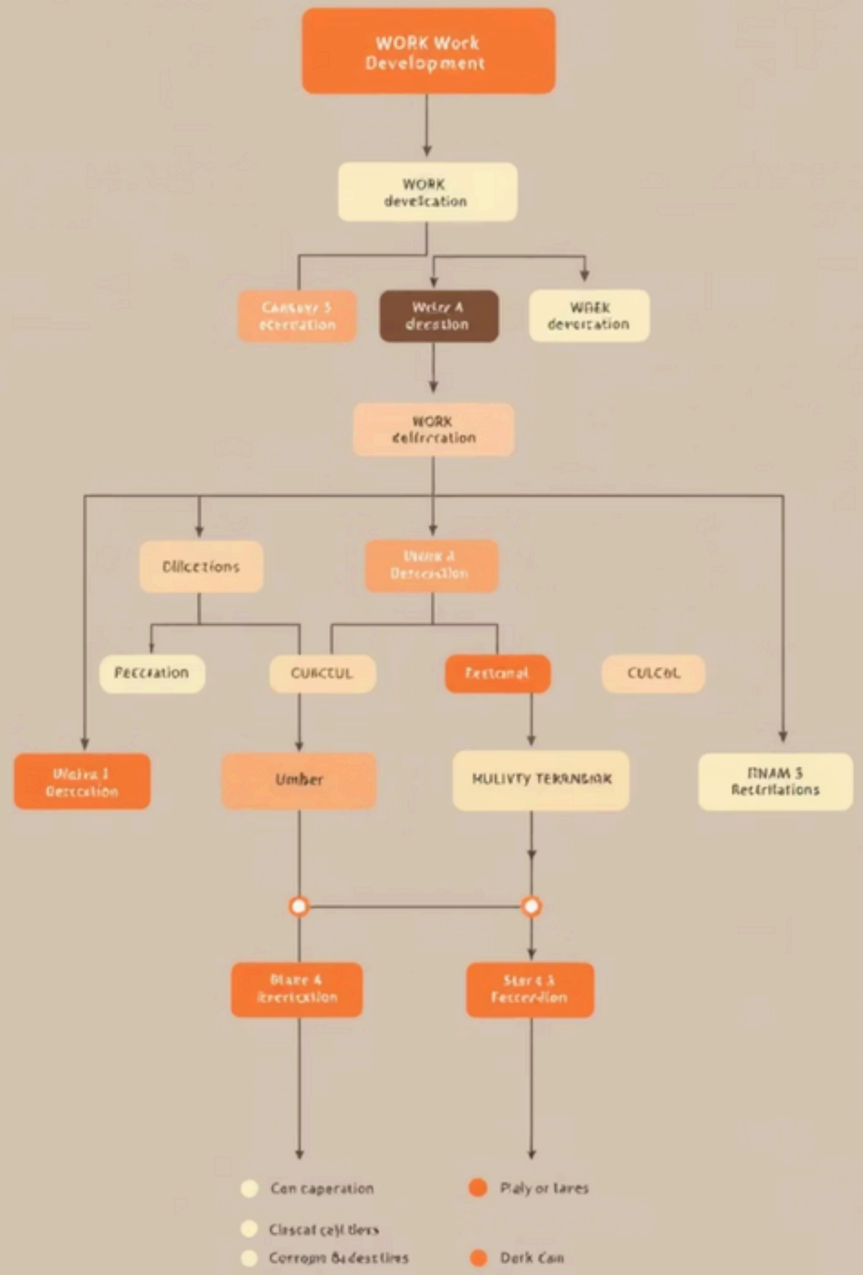
- 2.1 UI/UX Tasarımı (Wireframe Oluşturma, Renk Şeması Belirleme)
- 2.2 Veritabanı Tasarımı (Şema Oluşturma, İlişkiler Tanımlama)

## 3.0 Geliştirme

- 3.1 Frontend Kodlama (HTML/CSS, JavaScript Entegrasyonu)
- 3.2 Backend Kodlama (API Geliştirme, Sunucu Kurulumu)

## 4.0 Test ve Dağıtım

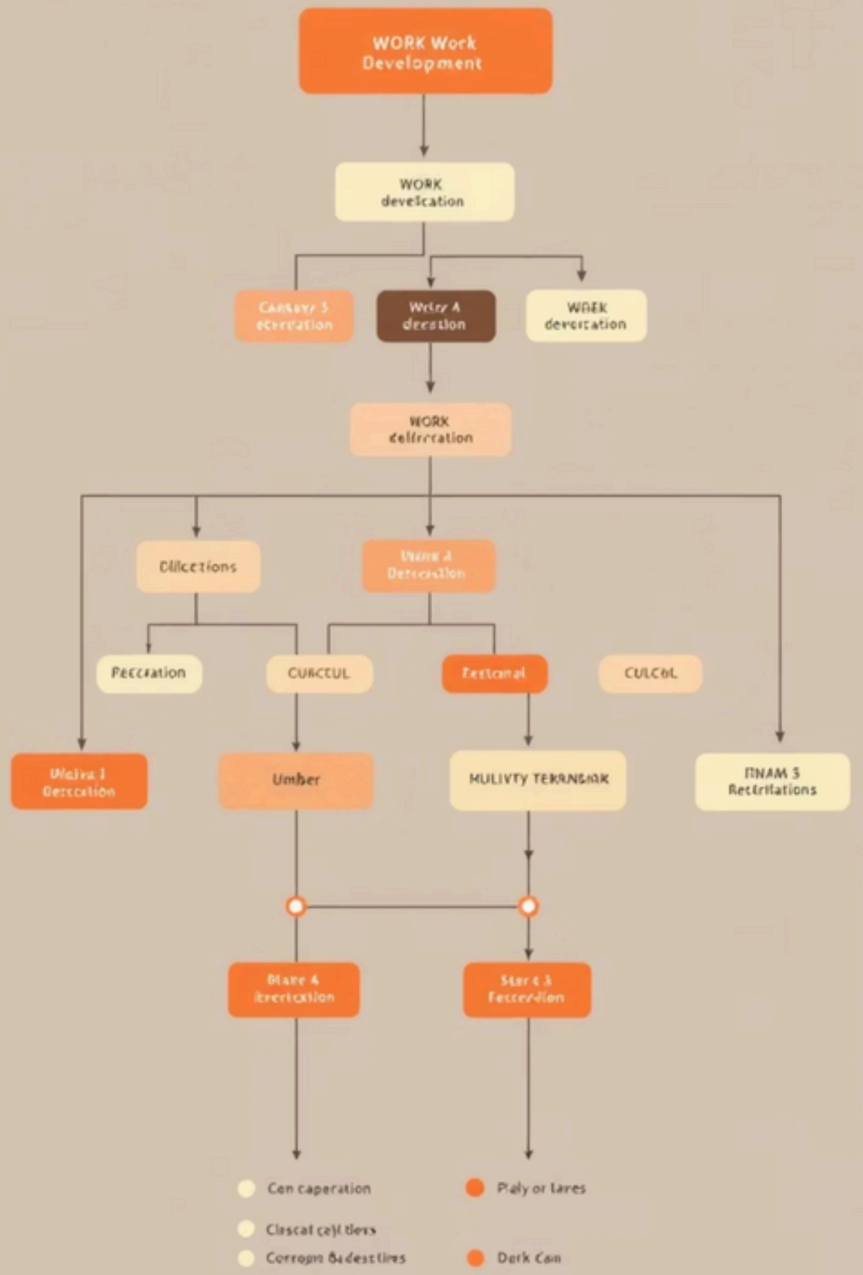
(Birim Testleri, Kullanıcı Kabul Testi, Canlıya Alma)




**Bu yapı, projenin 1000 saatlik bir işi 50'ye indirgeyerek yönetilebilir kılar.**

# UYGULAMA

## BU WEB SİTESİ İÇİN WBS YAPISINI TASARLAYIN.



# 2. Gantt Chart - Gantt Şeması

Gantt Chart, projenin zaman çizelgesini görselleştiren bir çubuk grafik türüdür. Yatay eksende zaman (gün, hafta, ay), dikey eksende ise WBS'den gelen görevler yer alır. Her görev için başlangıç/bitiş tarihleri, süreler ve bağımlılıklar (örneğin, Görev B, Görev A'dan sonra başlar) gösterilir. 1910'larda Henry Gantt tarafından geliştirilmiştir. Temel amacı: Görevlerin zamanlamasını izlemek, gecikmeleri önlemek ve kaynak çakışmalarını tespit etmek.

- WBS'den görevleri listeleyin
- Her göreve süre ve bağımlılık atayın
  (örneğin, Kritik Yol Yöntemi ile en uzun yol belirleyin)
- Zaman çizelgesini çizin
  (çubuklar süreleri temsil eder)
- Milestones ekleyin
  (örneğin, Tasarım Tamamlandı)

Görsel olarak kolay anlaşılır

İlerleme takibi için idealdir
(yüzde tamamlanma ile)

Ekip motivasyonunu artırır
(görünür ilerleme)



# Örnek: Web sitesi projesi için Gantt Chart

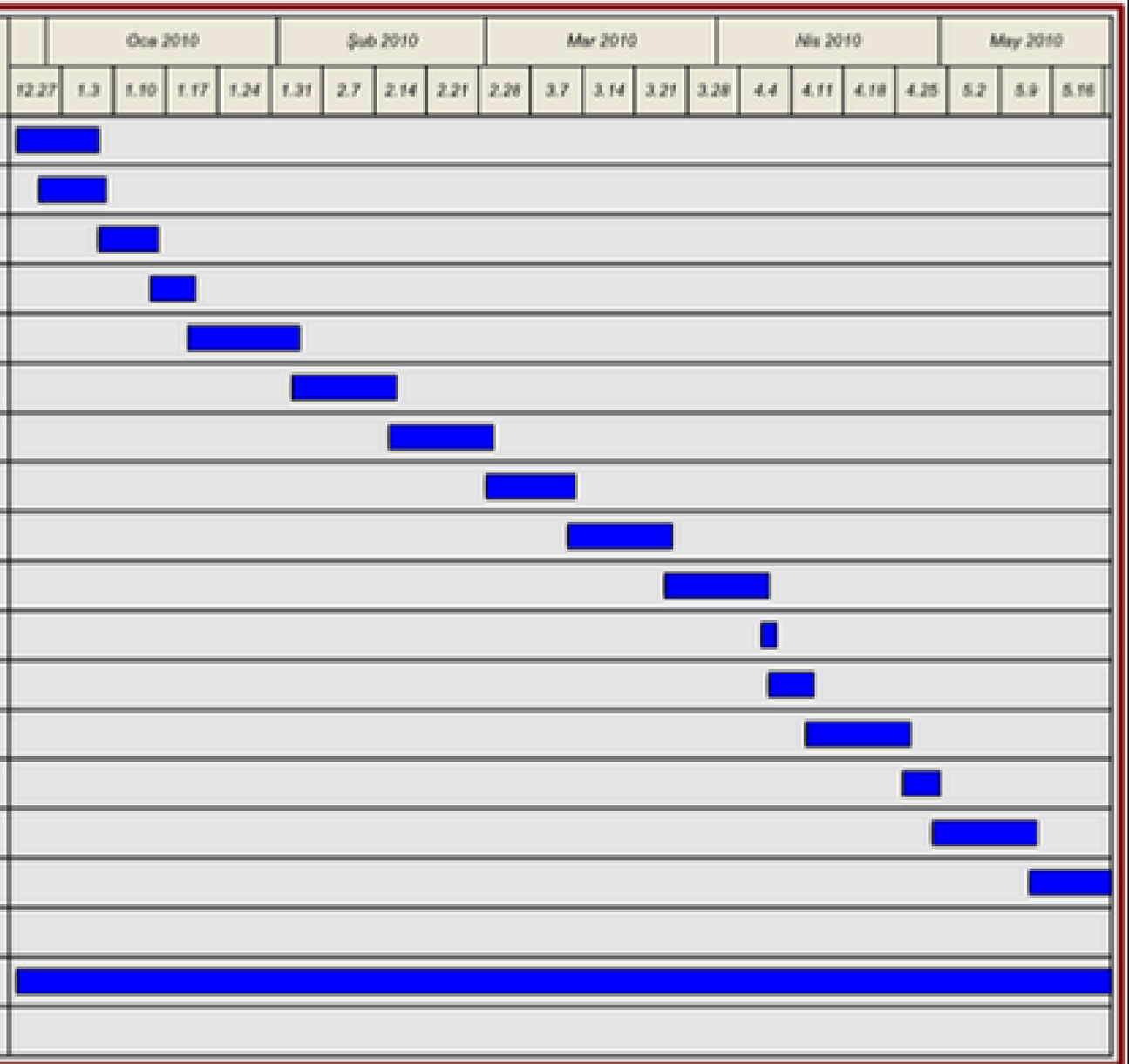
Aynı web sitesi projesi için basit bir Gantt Chart taslağı (metin tabanlı temsil; gerçekte Excel veya yazılımda çizilir):

| Görev | Süre (Hafta) | Başlangıç | Bitiş | Bağımlılık |
|---|---|---|---|---|
| Proje Tanımı | 1 | Hafta 1 | Hafta 1 | - |
| UI/UX Tasarımı | 2 | Hafta 2 | Hafta 3 | Proje Tanımı |
| Veritabanı Tasarımı | 1 | Hafta 2 | Hafta 2 | Proje Tanımı |
| Frontend Kodlama | 3 | Hafta 4 | Hafta 6 | UI/UX Tasarımı |
| Backend Kodlama | 3 | Hafta 4 | Hafta 6 | Veritabanı Tasarımı |
| Testler | 2 | Hafta 7 | Hafta 8 | Frontend & Backend |
| Canlıya Alma | 1 | Hafta 9 | Hafta 9 | Testler |

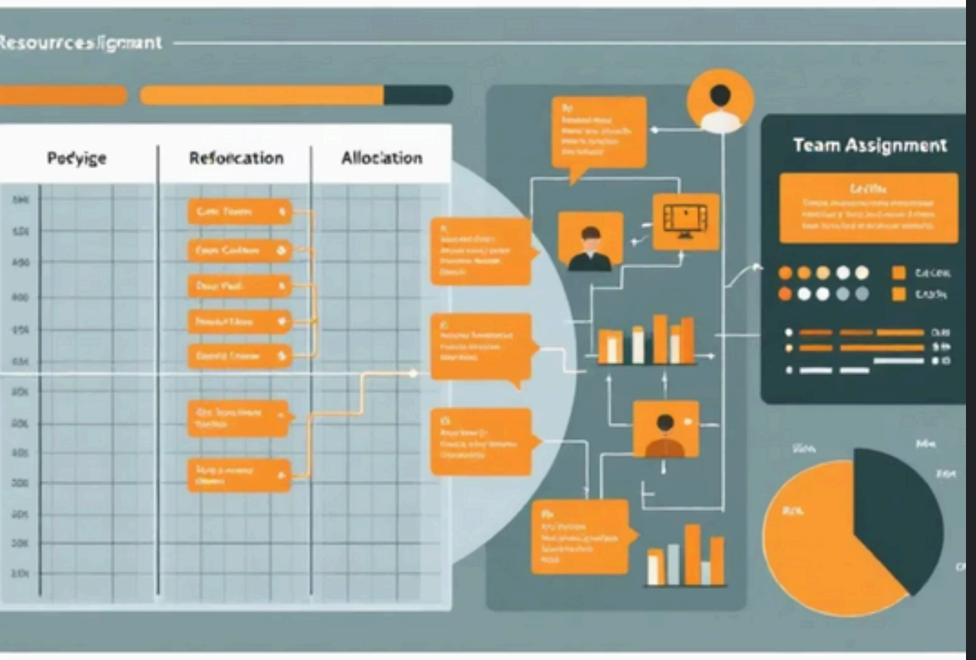
Bu chart'ta, toplam proje süresi 9 hafta olur ve Frontend/Backend paralel çalışabilir (kaynak tasarrufu).

# Örnek: Gantt Chart 2

| ID | Task Name | Start | Finish | Duration |
|---|---|---|---|---|
| 1 | Proje analizi | 28.12.2009 | 7.01.2010 | 9d |
| 2 | Veritabanı Diyagramının Oluşturulması | 31.12.2009 | 8.01.2010 | 7d |
| 3 | Classların tasarlanması | 8.01.2010 | 15.01.2010 | 6d |
| 4 | Web sitesi masterPage ve arayüz tasarımının yapılması | 15.01.2010 | 20.01.2010 | 4d |
| 5 | UserManagement modülünün hazırlanması | 20.01.2010 | 3.02.2010 | 11d |
| 6 | Forum modülü- soru sor/cevapla formlarının oluşturulması | 3.02.2010 | 16.02.2010 | 10d |
| 7 | Forum modülü- soru/cevap değerlendir bölümünün oluşturulması | 16.02.2010 | 1.03.2010 | 10d |
| 8 | Forum Bölümü-Çeviri Yap bölümünün oluşturulması | 1.03.2010 | 12.03.2010 | 10d |
| 9 | Forum modülü-Soru ve cevap arama motorunun geliştirilmesi | 12.03.2010 | 25.03.2010 | 10d |
| 10 | Makale Modülü- Makale oluştur formunun oluşturulması | 25.03.2010 | 7.04.2010 | 10d |
| 11 | Makale Modülü- Makale yorumla bölümünün oluşturulması | 7.04.2010 | 8.04.2010 | 2d |
| 12 | Makale Modülü- Makale değerlendir bölümünün oluşturulması | 8.04.2010 | 13.04.2010 | 4d |
| 13 | Makale arama motorunun geliştirilmesi | 13.04.2010 | 26.04.2010 | 10d |
| 14 | Program menülerinin oluşturulması | 26.04.2010 | 30.04.2010 | 5d |
| 15 | Anket yönetimi uygulamasının geliştirilmesi | 30.04.2010 | 13.05.2010 | 10d |
| 16 | Haberler yönetimi uygulamasının geliştirilmesi | 13.05.2010 | 26.05.2010 | 10d |
| 17 | Uygualam test evresi | 24.05.2010 | 04.06.2010 | 10d |
| 18 | Tez dökümantasyonunun yaratılması | 28.12.2009 | 4.06.2010 | 115d |
| 19 | Tez savunması | 24.05.2010 | 24.05.2010 | 1d |






# 3. Kaynak Tahsisi (Resource Allocation)

Kaynak tahsisi, proje için gerekli insan (ekip üyeleri), malzeme (yazılım lisansları), zaman ve bütçeyi görevlere dağıtan bir tekniktir. Aşırı yüklenmeyi (over-allocation) önler, verimliliği maksimize eder. Yazılım mühendisliğinde, geliştiricilerin becerilerine göre (örneğin, frontend uzmanı) atama yapılır; matris tabanlı (kaynak-görev matrisi) yaklaşımlar kullanılır. Faydası: Maliyetleri %15-25 düşürür, tükenmişliği önler; RACI matrisi (Responsible, Accountable, Consulted, Informed) ile roller netleşir.

# Kaynak Türleri

## İnsan Kaynakları

Ekip üyeleri (geliştiriciler, testçiler, proje yöneticileri). Beceri bazlı atama yapılır; örneğin, bir mobil app projesinde iOS uzmanı geliştiriciye iOS modülü verilir.

## Malzeme ve Ekipman

Yazılım lisansları (örneğin, Adobe XD için 500 TL lisans), sunucu kiralama (AWS hesabı) veya donanım (test cihazları).

## Bütçe

Görev bazlı maliyet dağılımı (örneğin, kodlama için 20.000 TL, test için 5.000 TL).

## Zaman

Görev süreleri ile kaynak yükü (örneğin, bir geliştirici haftada 40 saat çalışabilir, %100 yük = tam zamanlı).

# Temel Kavramlar:

- Over-Allocation (Aşırı Yükleme): Bir kaynağın kapasitesini aşması (örneğin, bir geliştirici aynı anda 3 göreve %50 yükle atanır, toplam %150 olur). Bu, tükenmişlik (burnout) ve gecikmelere yol açar.

- Resource Leveling (Kaynak Yumuşatma): Çakışmaları gidermek için görevleri geciktirme veya kaynak ekleme.

- Under-Allocation (Yetersiz Yükleme): Kaynakların boşta kalması, maliyetleri artırır.

- Matris Yaklaşımları: Kaynak-görev matrisi (Resource Assignment Matrix - RAM) kullanılır; satırlar kaynaklar, sütunlar görevler, kesişimlerde yük yüzdeleri (% veya saat) belirtilir.



# Takım Yönetimi ve İletişim Stratejileri:

Proje yönetimi sürecinin kalbi, elbette insanlardır. Bir projenin teknik araçları (WBS, Gantt gibi) ne kadar kusursuz olursa olsun, ekibin dinamiklerini yönetmek, rollerini netleştirmek, motivasyonlarını yüksek tutmak ve iletişim kanallarını akıcı hale getirmek, başarıyı %70 oranında belirleyen unsurdur (PMI raporlarına göre). Neden mi? Çünkü en iyi plan bile, motivasyonsiz bir ekipte çöker; net olmayan roller ise kaosa yol açar, çatışmalar ise verimliliği %30'a kadar düşürebilir.

# Roller: Takım Üyelerinin Pozisyonları ve Dinamikleri

Roller, takımın "kim ne yapar?" sorusuna cevap verir. Proje başlangıcında tanımlanmazsa, kimin neyi yapacağı belirsiz kalır ve verimsizlik artar.

## Fonksiyonel Roller (Görev Bazlı):

### Proje Yöneticisi (PM)

Genel koordinatör. Planı yönetir, paydaşlarla iletişim kurar, riskleri izler. Örnek: CRM projemizde PM, haftalık raporları hazırlar ve bütçeyi denetler.

### Teknik Lider/Geliştirici

Kod yazar, sorun çözer. Alt roller: Frontend (UI/UX), Backend (veri), Full-Stack (her şey).

### Test/QA Uzmanı

Kaliteyi sağlar, hataları bulur.

### Analist/Tasarımcı

Gereksinimleri toplar, prototipler çizer.

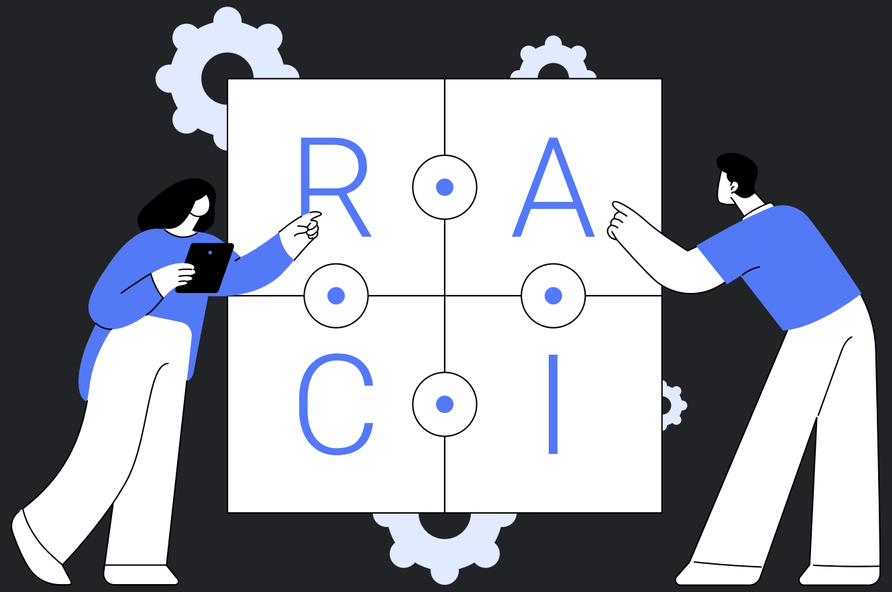
### Destek Rolleri

İletişimci (paydaş ilişkileri), Dokümantasyon Uzmanı (raporlar).

# RACI Matrisi: Sorumluluk ve İletişim Netliği

RACI matrisi, "kim ne için sorumlu?"yu görselleştiren bir araçtır. RACI, Responsible (Sorumlu), Accountable (Hesap Verebilir), Consulted (Danışılan), Informed (Bilgilendirilen) kısaltmasıdır. Temel bir araçtır ve rol belirsizliğini %40 azaltır. Matris, satırlarda görevler (WBS'den), sütunlarda ekip üyeleri yer alır; her kesişimde R/A/C/I harfi yazılır.



## Responsible (R)

Görevi fiilen yerine getiren, yani işi yapan kişi veya kişilerdir. Bir görevin tamamlanabilmesi için en az bir 'R' atanması zorunludur. Karmaşık görevlerde birden fazla 'R' olabilir.

## Accountable (A)

Görevin tamamlanmasından nihai olarak sorumlu olan ve işin kalitesini onaylayan kişidir. Bu kişi, görevin sahibi olarak kabul edilir ve son kararı verme yetkisine sahiptir. Karar süreçlerinde belirsizliği ve tıkanıklığı önlemek için her görevde sadece bir tane 'A' olabilir. Bu, RACI matrisinin en önemli kurallarından biridir.

## Consulted (C)

Görevle ilgili uzmanlığına veya bilgisine başvurulan, görüşü alınan kişi veya gruplardır. Bu roldeki kişilerle iki yönlü bir iletişim söz konusudur; hem bilgi verirler hem de geri bildirimde bulunurlar.

## Informed (I)

Sürecin ilerleyişi, tamamlanan görevler veya alınan kararlar hakkında tek yönlü olarak bilgilendirilen kişi veya gruplardır. Bu kişilerin sürece doğrudan bir katkısı veya geri bildirimi beklenmez; sadece gelişmelerden haberdar edilmeleri gerekir.

# Örnek: RACI Matrisi

| Görev | PM | Ali (Backend) | Ayşe (Frontend) | Mehmet (Full-Stack) | Deniz (Testçi) |
|---|---|---|---|---|---|
| Gereksinim Analizi | A/R | C | R | I | I |
| Tasarım | A | R | R | C | I |
| Geliştirme | A | R | R | R | C |
| Test | A | I | I | C | R |
| Dağıtım | A/R | I | C | R | I |

- Açıklama: Gereksinim Analizi'nde PM hem sorumlu (R) hem hesap verebilir (A), Ayşe yapar (R), Ali danışılır (C). Bu, toplantılarda kimin konuşacağını netleştirir.
- İletişim Entegrasyonu: RACI, e-posta trafiğini azaltır; örneğin, I'ler sadece Slack bildirim alır.

Avantajlar: Çatışmaları önler (örneğin, "Bu benim işim değil" demeyi engeller), hesap verebilirliği artırır. Dezavantaj: Karmaşık projelerde matris büyür (50+ satır) – çözüm: Alt matrisler. PMI'ye göre, RACI kullanan ekipler %30 daha az hata yapar.



# Kök Neden Analizi (RCA)

**Kök Neden Analizi (Root Cause Analysis - RCA), bir sorunun yüzeysel semptomlarını değil, arkasındaki gerçek sebepleri (kök nedenleri) tarafsız ve analitik bir şekilde ortaya çıkaran bir yöntemdir. Amaç: Problemi kalıcı olarak çözmek, maskeleme yerine sürdürülebilir çözümler üretmek. Suçlama odaklı değil, "neden?" sorusunu sürekli sorarak ilerler; ekip motivasyonunu korur, öğrenmeyi teşvik eder. Özellikle proje yönetimi, kalite kontrolü veya krizlerde kullanılır. Örneğin, gecikme veya hata durumunda %80 sorun %20 kök nedenden kaynaklanır**

# Balık Kılçığı Yöntemiyle Kök Neden Analizi

## Problem Tanımlama

İlk önce problem tanımlanır ve diyagramın sonuç kısmına yazılır.

## Temel Faktörleri Belirleme

Probleme etki edebilecek temel faktörler kılçıklara kutucuk şeklinde sıralanır.

## Nedenleri Yazma

Her bir faktörde sonuca etki edebilecek nedenler yazılır. Söz konusu nedenlerin asıl kaynağının tespiti için ard arda niçin sorusu sorularak her bir faktörde probleme yol açan temel kök neden tespit edilmeye çalışılır.

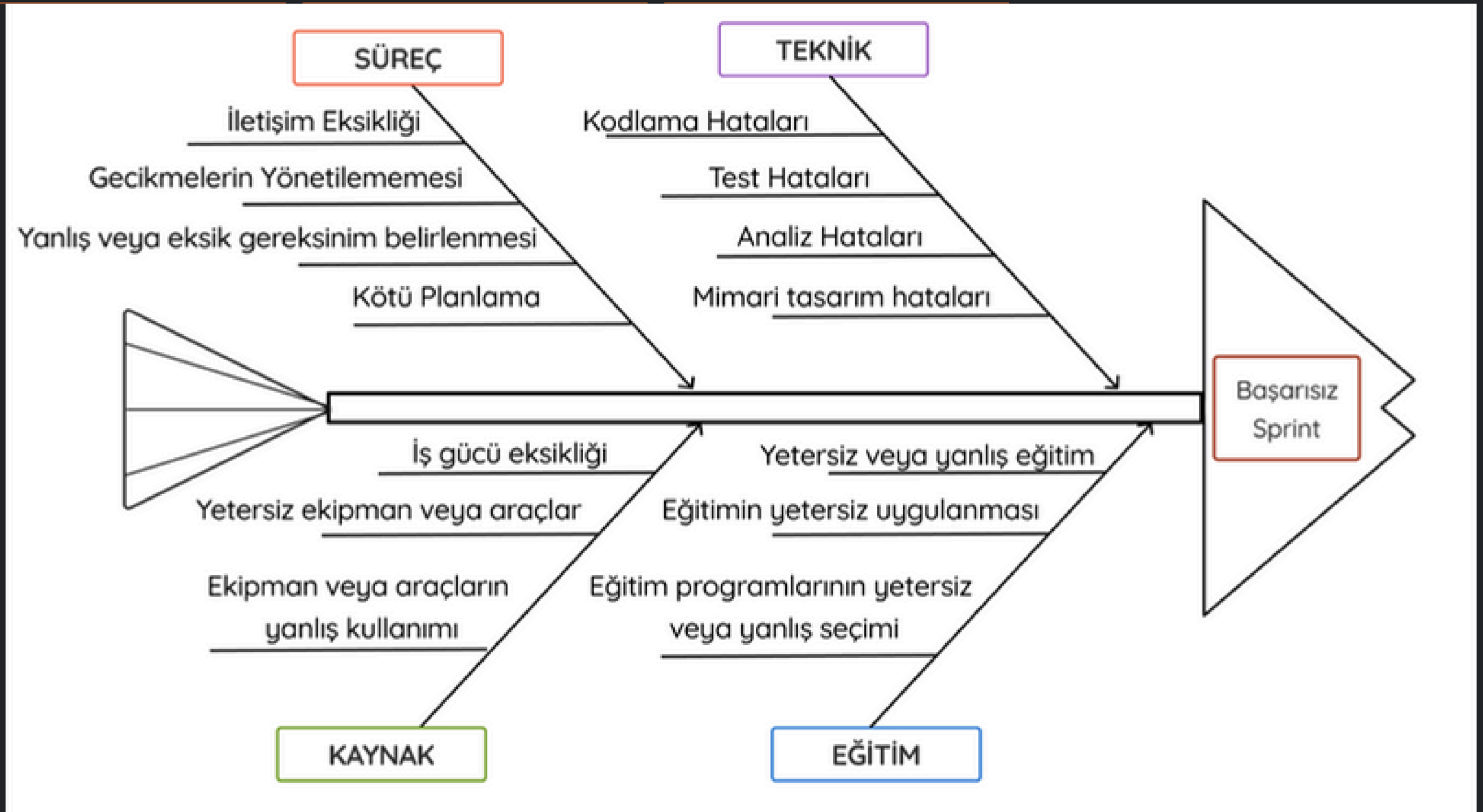
## Görselleştirme

Tüm bu çalışma sonucunda, problem şeklinde ortaya çıkmış bir olay ile bu problemin nedenleri ayrıntılı ve kategorize edilmiş bir şekilde açıkça görünür hale gelir.

## Önceliklendirme

Ayrıntılı nedenler üzerinden önceliklendirme yapılarak hangi nedenler için ilk planda tedbir geliştirileceği kararlaştırılır.

# Balık Kılçığı Yöntemiyle Kök Neden Analizi



# Gerçek Hayat Örneği: Fabrika Yangını

- Fabrika müdürü günlük olağan fabrika turunu atarken yerde birkaç yağ damlası görür. Temizlikçiyi çağırır herkesin içerisinde azarlar, hemen temizlemesini söyler ve gider. Temizlikçi temizler fakat yine yağ damlasını görür, korkar sürekli yağı temizlemeye devam eder. Sonra birgün sızıntı artar ve tavandaki borular patlayarak yangın çıkmasına neden olmuştur.

- Oysa müdür temizlikçiyi değil de ustabaşını çağırsa ve problemin kaynağını öğrenirdi.  Bu hikayede kök neden müdürdür.

